

Ecosistema de Automatización IA Autónomo para la Gestión de Tickets de Soporte de RiDMi

Entrega Final – Coderhouse

Proyecto: Sistema inteligente de soporte estudiantil RiDMi

Estudiante: Valentina Morey

Orquestador: Make

Base de datos: Airtable

Procesamiento de IA: OpenAI

Canal de salida: Gmail

Fecha: 31 de julio de 2026

Resumen ejecutivo

El presente proyecto desarrolla un ecosistema de automatización con inteligencia artificial para gestionar de extremo a extremo los tickets de soporte de RiDMi, una plataforma digital orientada a estudiantes.

La solución integra Make como orquestador del flujo, Airtable como base de datos y memoria operativa, OpenAI como motor de clasificación y generación de respuestas, y Gmail como canal de comunicación con los estudiantes.

Cuando se registra un nuevo ticket con estado “Pendiente”, Make recupera la información almacenada en Airtable y la procesa mediante un prompt dinámico enviado a OpenAI. El modelo analiza el asunto y el mensaje del estudiante para determinar la categoría, la prioridad y el sentimiento, y genera una propuesta de respuesta personalizada.

El sistema incorpora un punto de validación humana o *Human-in-the-loop*. Antes de enviar una respuesta al estudiante, el flujo se detiene en el estado “Esperando aprobación”. Una persona responsable revisa la respuesta generada por IA y autoriza el envío mediante un cambio de estado. Una vez aprobada, Make envía el correo a través de Gmail y registra en Airtable el estado final, la fecha de envío y el **Gmail Thread ID**.

La arquitectura también incluye una ruta de manejo de errores. Si se produce una falla durante la comunicación con la API de OpenAI, el sistema evita que el flujo continúe incorrectamente, actualiza el ticket y crea un registro técnico asociado en Airtable. Este registro permite conocer el módulo afectado, el tipo de error, el mensaje recibido, la cantidad de intentos y el estado de resolución.

Finalmente, se creó un dashboard de control en Airtable que permite visualizar los principales indicadores del proceso: cantidad total de tickets, tickets enviados, tickets recientes, errores registrados y distribuciones por categoría, prioridad y sentimiento.

El resultado es un sistema trazable, escalable y resiliente que reduce tareas manuales, mejora la velocidad de atención y conserva el control humano antes de ejecutar una acción crítica.

Problema de negocio

RiDMi recibe consultas de estudiantes relacionadas con dificultades técnicas, cuestiones administrativas y necesidades de orientación académica. La gestión manual de estos mensajes presenta distintos desafíos:

- Los tickets deben revisarse y clasificarse individualmente.
- No existe una priorización automática de los casos urgentes.
- La redacción manual de cada respuesta consume tiempo.
- Las consultas pueden quedar sin seguimiento.
- Los errores técnicos no siempre quedan registrados de manera estructurada.
- La información se encuentra dispersa y dificulta el análisis global del servicio.
- El envío inmediato de respuestas generadas por IA podría producir comunicaciones incorrectas o descontextualizadas.

Frente a este problema, se diseñó un ecosistema capaz de interpretar lenguaje natural, organizar los tickets, proponer respuestas y mantener la trazabilidad completa del proceso.

La automatización no reemplaza totalmente la decisión humana. La inteligencia artificial se utiliza para asistir en la clasificación y elaboración de respuestas, mientras que una persona conserva la responsabilidad de aprobar la comunicación antes de que llegue al estudiante.

Objetivo general

Diseñar e implementar un ecosistema de automatización con inteligencia artificial que permita recibir, analizar, clasificar, responder y monitorear tickets de soporte estudiantil de RiDMi, integrando una base de datos dinámica, procesamiento mediante un modelo de lenguaje, validación humana, envío por correo electrónico y manejo controlado de errores.

Objetivos específicos

- Centralizar en Airtable la información de estudiantes, tickets, configuración de IA y errores técnicos.
- Automatizar la detección de nuevos tickets mediante un disparador inteligente en Make.
- Clasificar cada consulta según su categoría, prioridad y sentimiento.
- Generar una propuesta de respuesta personalizada mediante OpenAI.
- Utilizar variables dinámicas provenientes de Airtable y evitar información fija dentro del flujo.
- Incorporar un punto de aprobación humana antes del envío del correo.
- Enviar las respuestas aprobadas mediante Gmail.

- Registrar el estado final, la fecha de envío y el **Gmail Thread ID**.
- Incorporar una ruta alternativa para manejar fallas de la API de IA.
- Guardar los errores técnicos en una tabla vinculada al ticket correspondiente.
- Evitar reprocesamientos y posibles bucles mediante filtros por estado.
- Monitorear los resultados mediante un dashboard de Airtable.
- Evaluar el comportamiento del sistema mediante pruebas funcionales y una prueba controlada de error.

Alcance del proyecto

El proyecto comprende el ciclo completo de atención de un ticket:

1. Registro del ticket en Airtable.
2. Detección automática de registros pendientes.
3. Recuperación de la configuración activa de IA.
4. Análisis del mensaje mediante OpenAI.
5. Clasificación por categoría, prioridad y sentimiento.
6. Generación de una respuesta propuesta.
7. Cambio de estado a “Esperando aprobación”.
8. Revisión y aprobación humana.
9. Envío del correo mediante Gmail.
10. Registro de fecha de envío y **Gmail Thread ID**.
11. Actualización del estado a “Enviado”.
12. Registro técnico y detención controlada ante fallas.
13. Visualización de indicadores en el dashboard.

No forma parte del alcance actual la recepción automática de respuestas posteriores dentro del mismo hilo de Gmail ni la resolución autónoma de errores. Estas funciones se consideran posibles mejoras futuras.

Tecnologías utilizadas

Categoría obligatoria	Tecnología	Función
Orquestación	Make	Ejecuta la lógica, los filtros, las rutas y las actualizaciones
Base de datos	Airtable	Almacena estudiantes, tickets, configuración y errores
Procesamiento IA	OpenAI	Clasifica las consultas y genera respuestas propuestas

Canal de salida	Gmail	Envía las respuestas aprobadas
Monitoreo	Airtable Interfaces	Presenta KPIs, gráficos y filtros
Respaldo técnico	Blueprint de Make	Permite revisar y reutilizar la arquitectura del flujo

La elección de estas herramientas responde a la necesidad de construir una solución de bajo costo, visual, modular y adecuada para una primera etapa operativa de RiDMi.

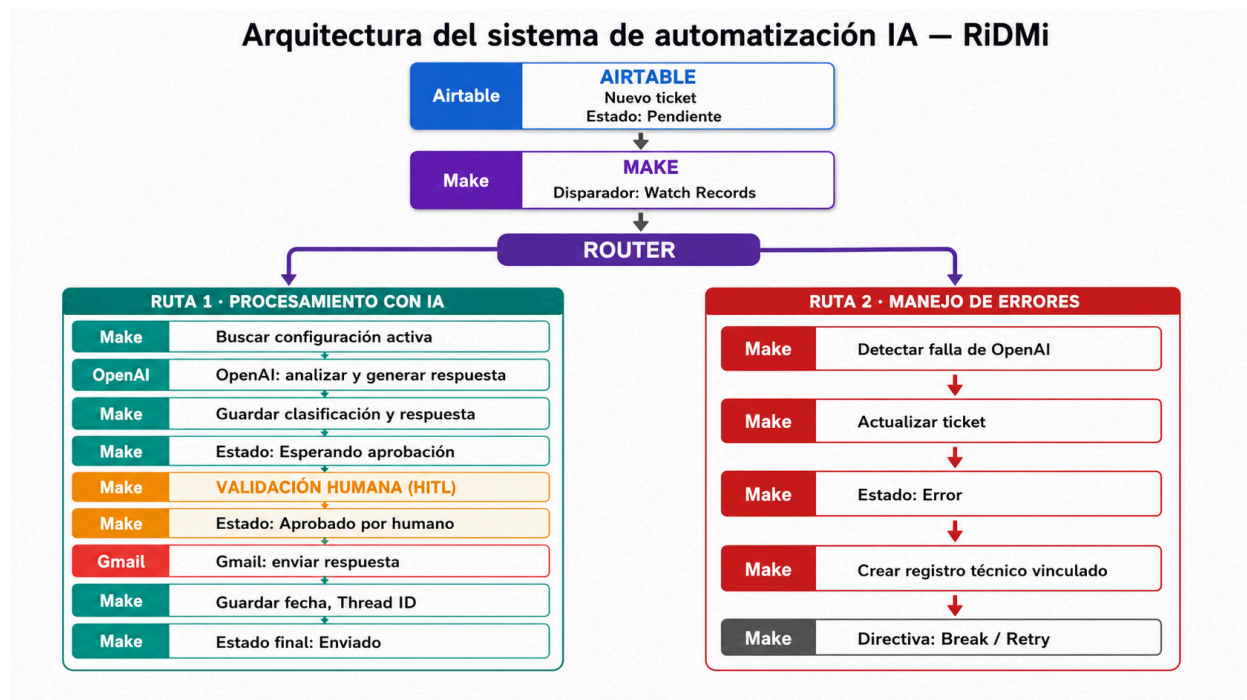
Mapa de arquitectura del sistema

El ecosistema de automatización de RiDMi utiliza una arquitectura modular en la que Airtable funciona como memoria central, Make coordina las operaciones, OpenAI procesa el lenguaje natural y Gmail ejecuta la comunicación final.

El sistema contiene dos recorridos principales:

- Camino exitoso: procesamiento, aprobación humana y envío.
- Camino de error: detección de una falla, registro técnico y detención controlada.

Diagrama general



Arquitectura general del ecosistema de automatización de tickets de soporte de RiDMi.

Componentes de la arquitectura

Componente	Tecnología	Responsabilidad
Fuente y memoria	Airtable	Almacenar tickets, estudiantes, configuración, estados y errores
Disparador	Make – Watch Records	Detectar tickets que cumplen las condiciones de ejecución
Control de rutas	Router de Make	Separar el procesamiento con IA del envío de tickets aprobados
Recuperación de configuración	Airtable – Search Records	Obtener el modelo, prompt y parámetros activos
Motor de IA	OpenAI	Clasificar el ticket y generar la propuesta de respuesta
Persistencia	Airtable – Update Record	Guardar los resultados y actualizar los estados
Validación humana	Airtable	Detener el proceso hasta que una persona apruebe la respuesta
Canal de salida	Gmail	Enviar la respuesta aprobada al estudiante
Trazabilidad	Gmail y Airtable	Guardar el Thread ID , las fechas y el estado final
Manejo de errores	Error Handler de Make	Capturar fallas y evitar que el escenario continúe incorrectamente
Registro técnico	Airtable – Registro de errores	Conservar diagnóstico, intentos y relación con el ticket
Monitoreo	Airtable Interfaces	Visualizar indicadores, distribuciones y tickets con errores

Flujo de información

Airtable actúa como fuente única de datos del sistema. Make consulta y actualiza los registros, mientras que OpenAI y Gmail reciben únicamente la información necesaria para cumplir su función.

La información circula de la siguiente manera:

1. Airtable entrega a Make los datos del ticket pendiente.
2. Make recupera la configuración activa de inteligencia artificial.
3. Make construye una solicitud dinámica y la envía a OpenAI.
4. OpenAI devuelve una clasificación y una respuesta propuesta.
5. Make guarda los resultados en el ticket.
6. El sistema espera la aprobación humana.
7. Tras la aprobación, Make envía la respuesta mediante Gmail.
8. Gmail devuelve el identificador del hilo.
9. Make registra el **Gmail Thread ID**, la fecha de envío y el estado final.
10. Si OpenAI falla, la ruta de error guarda un registro técnico y detiene el procesamiento.

Funcionamiento del flujo principal en Make

El escenario de Make se diseñó para ejecutar dos etapas diferentes dentro del ciclo de vida del ticket: procesamiento mediante inteligencia artificial y envío posterior a la aprobación humana.

Disparador inteligente

El escenario comienza con el módulo **Watch Records** de Airtable. Este módulo supervisa los registros relevantes y permite que el flujo procese únicamente los tickets que cumplen las condiciones definidas.

El uso de estados evita que todos los registros sean procesados en cada ejecución. De esta manera, se reduce el consumo innecesario de operaciones y se previenen reprocesamientos.

Los estados principales utilizados son:

Pendiente
Esperando aprobación
Aprobado por humano
Enviado
Error

Cada estado representa una etapa concreta y determina qué acción puede realizarse a continuación.

Separación de rutas

Después del disparador, un Router divide el escenario en rutas con responsabilidades diferentes.

Ruta de procesamiento con IA

Esta ruta se ejecuta cuando el ticket está en estado **Pendiente**.

Sus operaciones son:

1. Recuperar el ticket y los datos relacionados con el estudiante.
2. Buscar en Airtable la configuración activa de OpenAI.
3. Construir el prompt utilizando variables dinámicas.
4. Enviar el asunto y el mensaje original al modelo.
5. Recibir la categoría, prioridad, sentimiento y respuesta propuesta.
6. Guardar los resultados en Airtable.
7. Cambiar el estado a **Esperando aprobación**.

Ruta de envío

Esta ruta se ejecuta cuando el ticket se encuentra en estado **Aprobado por humano**.

Sus operaciones son:

1. Recuperar la respuesta autorizada.
2. Obtener el correo relacionado con el estudiante.
3. Enviar el mensaje mediante Gmail.
4. Obtener el identificador del hilo enviado.
5. Generar la fecha actual.
6. Guardar el **Gmail Thread ID**.
7. Guardar la fecha de envío.
8. Cambiar el estado a **Enviado**.

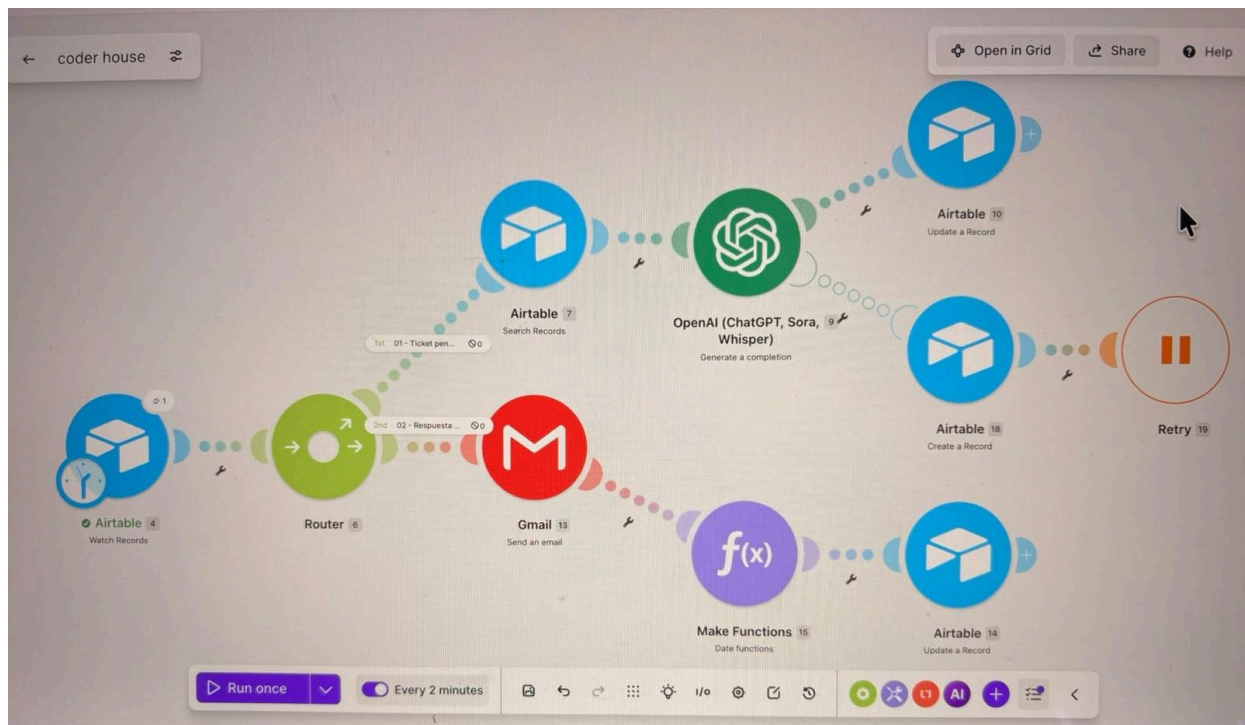
Esta separación impide que una respuesta recién generada por IA se envíe automáticamente sin revisión.

Módulos del escenario

El Blueprint contiene los siguientes módulos principales:

Orden	Módulo	Función
1	Airtable – Watch Records	Detectar tickets que deben procesarse
2	Router	Distribuir la ejecución según el estado
3	Airtable – Search Records	Recuperar la configuración activa de IA
4	OpenAI – Generate a Completion	Analizar el ticket y proponer una respuesta

5	Airtable – Update Record	Guardar la clasificación y el contenido generado
6	Gmail – Send an Email	Enviar la respuesta aprobada
7	Make Functions – Fecha actual	Generar la fecha de envío
8	Airtable – Update Record	Guardar el resultado final y la trazabilidad
9	Airtable – Create Record	Crear un registro cuando ocurre un error
10	Directiva de error	Reintentar o detener controladamente la ejecución



Escenario completo de automatización implementado en Make.

Control de reprocesamientos y bucles

El control mediante estados funciona como protección frente a bucles infinitos. Un ticket no permanece en el mismo estado después de completar una etapa:

Pendiente → Esperando aprobación

Aprobado por humano → Enviado

Falla de procesamiento → Error

Por lo tanto, después de una ejecución exitosa, el registro deja de cumplir la condición que activó esa ruta.

Los filtros comparan campos de tipo texto con opciones válidas del campo **Estado**. Para evitar inconsistencias, los valores deben mantenerse escritos exactamente de la misma manera tanto en Airtable como en los filtros de Make.

Estructura del “cerebro”: base de datos de Airtable

Airtable funciona como memoria operativa y fuente central de información. La base se diseñó con tablas vinculadas para evitar datos aislados y reducir la duplicación.

Las tablas principales son:

1. Estudiantes.
2. Tickets coder house.
3. Configuración IA.
4. Registro de errores.

Relaciones entre tablas

ESTUDIANTES



Un estudiante puede tener varios tickets



TICKETS CODER HOUSE



Un ticket puede tener uno o más registros de error



REGISTRO DE ERRORES

CONFIGURACIÓN IA



Proporciona parámetros dinámicos al escenario



PROCESAMIENTO EN MAKE Y OPENAI

Las relaciones permiten consultar los tickets de cada estudiante y asociar los errores técnicos con el registro que originó la ejecución.

Tabla Estudiantes

Esta tabla almacena la información mínima necesaria para identificar al estudiante y enviarle una respuesta.

Identificación	ID del ticket	Identificador único, por ejemplo TKT-0014
Entrada	Asunto	Resumen de la consulta
Entrada	Mensaje original	Texto escrito por el estudiante
Relación	Estudiante	Vínculo con la tabla Estudiantes
Control	Estado	Define la etapa del flujo
IA	Categoría IA	Clasificación temática
IA	Prioridad IA	Nivel de urgencia
IA	Sentimiento IA	Tono detectado en el mensaje
IA	Respuesta propuesta	Borrador generado por OpenAI
HITL	Respuesta aprobada	Versión autorizada para el envío
HITL	Aprobado por	Responsable de la validación
HITL	Fecha de aprobación	Momento de la autorización
Salida	Gmail Thread ID	Identificador del hilo enviado
Salida	Fecha de envío	Momento de la comunicación
Error	Registro de errores	Relación con fallas técnicas
Auditoría	Última modificación	Seguimiento de actividad reciente

Este diseño permite observar todo el ciclo de vida del ticket en un mismo registro.

RidMI							
Datos Automatizaciones Interfaces Formularios							
recursos de estudio prompt IA Estudiantes coder house proyecto Tickets coder house Registro de errores Configuración IA 1 tabla oculta Herramientas							
Grid view 2 campos ocultos Filtro Grupo IT Clasificar Configurar color Compartir y sincronizar							
ID ticket	Asunto	Mensaje original	Fecha de recepción	Estudiante	Gmail Thread ID	Gmail Message ID	
1 TKT-0001	problema al resumir un d...	Subí un documento, pero ...		01			
2 TKT-0002	problema al resumir un d...	Subí un documento, pero ...		02			
3 TKT-0003	problema al resumir un d...	Subí un documento, pero ...		03			
4 TKT-0004	La herramienta no genera...	Hola, subí un documento ...		01			
5 TKT-0005	No puedo ingresar a mi c...	Hola, intento iniciar sesió...		02			
6 TKT-0006	Ayuda para organizar el ...	Hola, estoy comenzando ...		03			
7 TKT-0007	No puedo descargar el tr...	Hola, necesito entregar m...		01			
8 TKT-0011	Prueba controlada del Err...	Hola, intento generar un r...		03			
9 TKT-0012	La herramienta no genera...	Hola, subí un documento ...		02			
10 TKT-0013	La herramienta no genera...	Hola, subí un documento ...		03	19fb3ea0050bbc47		
11 TKT-0014	No puedo visualizar el res...	Subí un documento para ...		01	19fb8e72b7c6f43a		

Ticket clasificado mediante IA.

RidMI							
Datos Automatizaciones Interfaces Formularios							
recursos de estudio prompt IA Estudiantes coder house proyecto Tickets coder house Registro de errores Configuración IA 1 tabla oculta Herramientas							
Grid view 2 campos ocultos Filtro Grupo IT Clasificar Configurar color Compartir y sincronizar							
ID ticket	Categoría IA	Prioridad IA	Sentimiento	Resumen IA	Información faltante	Respuesta...	Respu...
1 TKT-0001	Soporte técnico	Media	Neutral			Hola Valentina, lamento q...	
2 TKT-0002	Soporte técnico	Media	Neutral			Lamentamos que estés te...	
3 TKT-0003	Soporte técnico	Media	Neutral			Hola Barby Valentina, gra...	
4 TKT-0004	Soporte técnico	Alta	Negativo			Hola Valentina, lamento q...	
5 TKT-0005	Administrativo	Media	Neutral			Hola Topi, lamento que es...	
6 TKT-0006	Consulta académica	Media	Neutral			Hola Barby, claro que sí, t...	
7 TKT-0007	Soporte técnico	Alta	Negativo			Hola Valentina, lamento q...	
8 TKT-0011	Soporte técnico	Alta	Neutral			Hola Barby Valentina, gra...	
9 TKT-0012	Soporte técnico	Alta	Negativo			Hola Topi, lamento que es...	
10 TKT-0013	Soporte técnico	Alta	Negativo			Hola Barby, lamento que ...	
11 TKT-0014	Soporte técnico	Alta	Negativo			Hola Valentina, lamento q...	

RidMI

Datos Automatizaciones Interfaces Formularios

usuario Documentos bloques del documento recursos de estudio prompt IA Estudiantes coder house proyecto Tickets coder house 1 tabla oculta Herramientas

Grid view 2 campos ocultos Filtro Grupo Clasificar Configurar color Compartir y sincronizar

ID ticket	Respuesta aprobada	Estado	A Gmail Thread ID	A Aprobado por	Fecha de aprobación	Fecha de envío
1 TKT-0001	a, lamento q...	Enviado		Valentina Morey	29/7/2026	29/7/2026
2 TKT-0002	que estás te...	Enviado		Valentina Morey	29/7/2026	29/7/2026
3 TKT-0003	Valentina, gra...	Enviado		Valentina Morey	29/7/2026	29/7/2026
4 TKT-0004	a, lamento q...	Enviado		Valentina Morey	29/7/2026	29/7/2026
5 TKT-0005	nento que es...	Enviado		Valentina Morey	29/7/2026	29/7/2026
6 TKT-0006	largo que sí, t...	Enviado		Valentina Morey	29/7/2026	29/7/2026
7 TKT-0007	a, lamento q...	Enviado		Valentina Morey	29/7/2026	29/7/2026
8 TKT-0011	Valentina, gra...	Error		Valentina Morey	30/7/2026	30/7/2026
9 TKT-0012	nento que es...	Enviado		Valentina Morey	30/7/2026	30/7/2026
10 TKT-0013	mento que ...	Enviado	19fb3ea0050bbc47	Valentina Morey	30/7/2026	30/7/2026
11 TKT-0014	que estás te...	Enviado	19fb8e72b7c6f43a	Valentina Morey	31/7/2026	31/7/2026

+ Añadir...

RidMI

Datos Automatizaciones Interfaces Formularios

recursos de estudio prompt IA Estudiantes coder house proyecto Tickets coder house Registro de errores Configuración 1 tabla oculta Herramientas

Grid view 2 campos ocultos Filtro Grupo Clasificar Configurar color Compartir y sincronizar

ID ticket	e errores	ultima modificacion	nomb...	Nombre (from...	email	Email (from email)	Número de ticket
1 TKT-0001		29/7/2026 7:22pm	01		01	@gmail.com	1
2 TKT-0002		29/7/2026 7:41pm	02		02	@gmail.com	2
3 TKT-0003		29/7/2026 7:45pm	03		03	@gmail...	3
4 TKT-0004		29/7/2026 8:03pm	01		01	@gmail.com	4
5 TKT-0005		29/7/2026 8:05pm	02		02	@gmail.com	5
6 TKT-0006		29/7/2026 8:07pm	03		03	@gmail...	6
7 TKT-0007		29/7/2026 8:09pm	01		01	@gmail.com	7
8 TKT-0011		31/7/2026 12:37pm	03		03	@gmail...	11
9 TKT-0012		30/7/2026 1:27pm	02		02	@gmail.com	12
10 TKT-0013		30/7/2026 1:44pm	03		03	@gmail...	13
11 TKT-0014		31/7/2026 1:05pm	01		01	@gmail.com	14

+ Añadir...

11 registros Suma: 78

Registro de aprobación humana, envío y trazabilidad.

RidMI									
Datos Automizaciones Interfaces Formularios									
recursos de estudio prompt IA Estudiantes coder house proyecto Tickets coder house Registro de errores Configuración IA 1 tabla oculta Herramientas									
Grid view 2 campos ocultos Filtro Grupo IT Clasificar Configurar color Compartir y sincronizar									
	ID ticket	Resumen IA	Información faltante	Respuesta...	Respuesta aprobada	Estado	Aprobado por	Fecha de aprobación	
1	TKT-0001			Hola Valentina, lamento q...		Enviado			29/7
2	TKT-0002			Lamentamos que estés te...		Enviado			29/7
3	TKT-0003			Hola Barby Valentina, gra...		Enviado			29/7
4	TKT-0004			Hola Valentina, lamento q...		Enviado			29/7
5	TKT-0005			Hola Topi, lamento que es...		Enviado			29/7
6	TKT-0006			Hola Barby, claro que sí, t...		Enviado			29/7
7	TKT-0007			Hola Valentina, lamento q...		Enviado			29/7
8	TKT-0011			Hola Barby Valentina, gra...		Error			30/7
9	TKT-0012			Hola Topi, lamento que es...		Enviado			30/7
10	TKT-0013			Hola Barby, lamento que ...		Enviado			30/7
11	TKT-0014			Hola Valentina, lamento q...		Pendiente			31/7

RidMI									
Datos Automizaciones Interfaces Formularios									
usuario Documentos bloques del documento recursos de estudio prompt IA Estudiantes coder house proyecto Tickets coder house Re Lanzamiento Compartir									
Grid view Ocultar campos Filtro Grupo IT Clasificar Configurar color Compartir y sincronizar									
	ID ticket	ante	Respuesta...	Respuesta aprobada	Estado	Aprobado por	Fecha de aprobación	Fecha de envío	
1	TKT-0001		Hola Valentina, lamento q...		Enviado			29/7/2026	
2	TKT-0002		Lamentamos que estés te...		Enviado			29/7/2026	
3	TKT-0003		Hola Barby Valentina, gra...		Enviado			29/7/2026	
4	TKT-0004		Hola Valentina, lamento q...		Enviado			29/7/2026	
5	TKT-0005		Hola Topi, lamento que es...		Enviado			29/7/2026	
6	TKT-0006		Hola Barby, claro que sí, t...		Enviado			29/7/2026	
7	TKT-0007		Hola Valentina, lamento q...		Enviado			29/7/2026	
8	TKT-0011		Hola Barby Valentina, gra...		Error			30/7/2026	
9	TKT-0012		Hola Topi, lamento que es...		Enviado			30/7/2026	
10	TKT-0013		Hola Barby, lamento que ...		Enviado			30/7/2026	
	TKT-0014		Hola Valentina, lamento q...		Esperando aproba...			31/7/2026	

RidMI

Datos Automatizaciones Interfaces Formularios

usuario Documentos bloques del documento recursos de estudio prompt IA Estudiantes coder house proyecto Tickets coder house Re Lanzamiento Compartir

Grid view

Ocultar campos Filtro Grupo Clasificar Configurar color Compartir y sincronizar

ID ticket	ante	Respuesta...	Respuesta aprobada	Estado	Aprobado por	Fecha de aprobación	Fecha de envío
1 TKT-0001		Hola Valentina, lamento q...		Enviado			29/7/2026
2 TKT-0002		Lamentamos que estés te...		Enviado			29/7/2026
3 TKT-0003		Hola Barby Valentina, gra...		Enviado			29/7/2026
4 TKT-0004		Hola Valentina, lamento q...		Enviado			29/7/2026
5 TKT-0005		Hola Topi, lamento que es...		Enviado			29/7/2026
6 TKT-0006		Hola Barby, claro que sí, t...		Enviado			29/7/2026
7 TKT-0007		Hola Valentina, lamento q...		Enviado			29/7/2026
8 TKT-0011		Hola Barby Valentina, gra...		Error			30/7/2026
9 TKT-0012		Hola Topi, lamento que es...		Enviado			30/7/2026
10 TKT-0013		Hola Barby, lamento que ...		Enviado			30/7/2026
TKT-0014		Hola Valentina, lamento q...		Aprobado por hum...			31/7/2026

Ampliar

Añadir...

RidMI

Datos Automatizaciones Interfaces Formularios

is los cambios guardados Lanzamiento Compartir

usuario Documentos bloques del documento recursos de estudio prompt IA Estudiantes coder house proyecto Tickets coder house Re Lanzamiento Compartir

Grid view

Ocultar campos Filtro Grupo Clasificar Configurar color Compartir y sincronizar

ID ticket	ante	Respuesta...	Respuesta aprobada	Estado	Aprobado por	Fecha de aprobación	Fecha de envío
1 TKT-0001		Hola Valentina, lamento q...		Enviado			29/7/2026
2 TKT-0002		Lamentamos que estés te...		Enviado			29/7/2026
3 TKT-0003		Hola Barby Valentina, gra...		Enviado			29/7/2026
4 TKT-0004		Hola Valentina, lamento q...		Enviado			29/7/2026
5 TKT-0005		Hola Topi, lamento que es...		Enviado			29/7/2026
6 TKT-0006		Hola Barby, claro que sí, t...		Enviado			29/7/2026
7 TKT-0007		Hola Valentina, lamento q...		Enviado			29/7/2026
8 TKT-0011		Hola Barby Valentina, gra...		Error			30/7/2026
9 TKT-0012		Hola Topi, lamento que es...		Enviado			30/7/2026
10 TKT-0013		Hola Barby, lamento que ...		Enviado			30/7/2026
11 TKT-0014		Hola Valentina, lamento q...		Enviado			31/7/2026

Ampliar

Añadir...

11 registros

Tabla Configuración IA

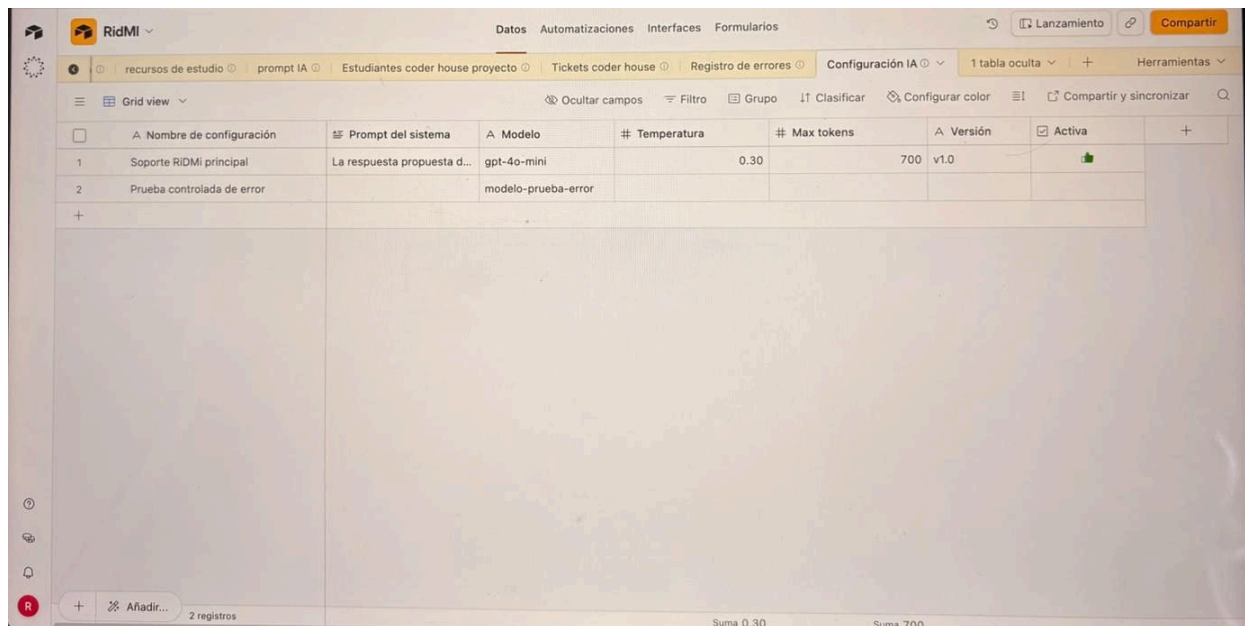
Esta tabla evita que parámetros importantes queden escritos de forma fija dentro del escenario.

Campo	Función
Nombre de configuración	Identificar la versión utilizada
Estado activo	Determinar qué configuración debe recuperar Make

Modelo	Seleccionar el modelo de OpenAI
Prompt del sistema	Definir el comportamiento esperado
Temperatura	Controlar la variabilidad de la respuesta
Máximo de tokens	Limitar la extensión y el costo
Categorías permitidas	Estandarizar la clasificación
Prioridades permitidas	Estandarizar el nivel de urgencia
Sentimientos permitidos	Estandarizar el análisis emocional

La configuración puede modificarse desde Airtable sin reconstruir el escenario. Esto mejora la mantenibilidad y permite probar modelos o prompts diferentes.

Configuración dinámica utilizada por el módulo de OpenAI.



	A Nombre de configuración	P Prompt del sistema	A Modelo	# Temperatura	# Max tokens	A Versión	Activa	
1	Soporte RIDMI principal	La respuesta propuesta d...	gpt-4o-mini	0.30	700	v1.0		
2	Prueba controlada de error		modelo-prueba-error					
+								

Tabla Registro de errores

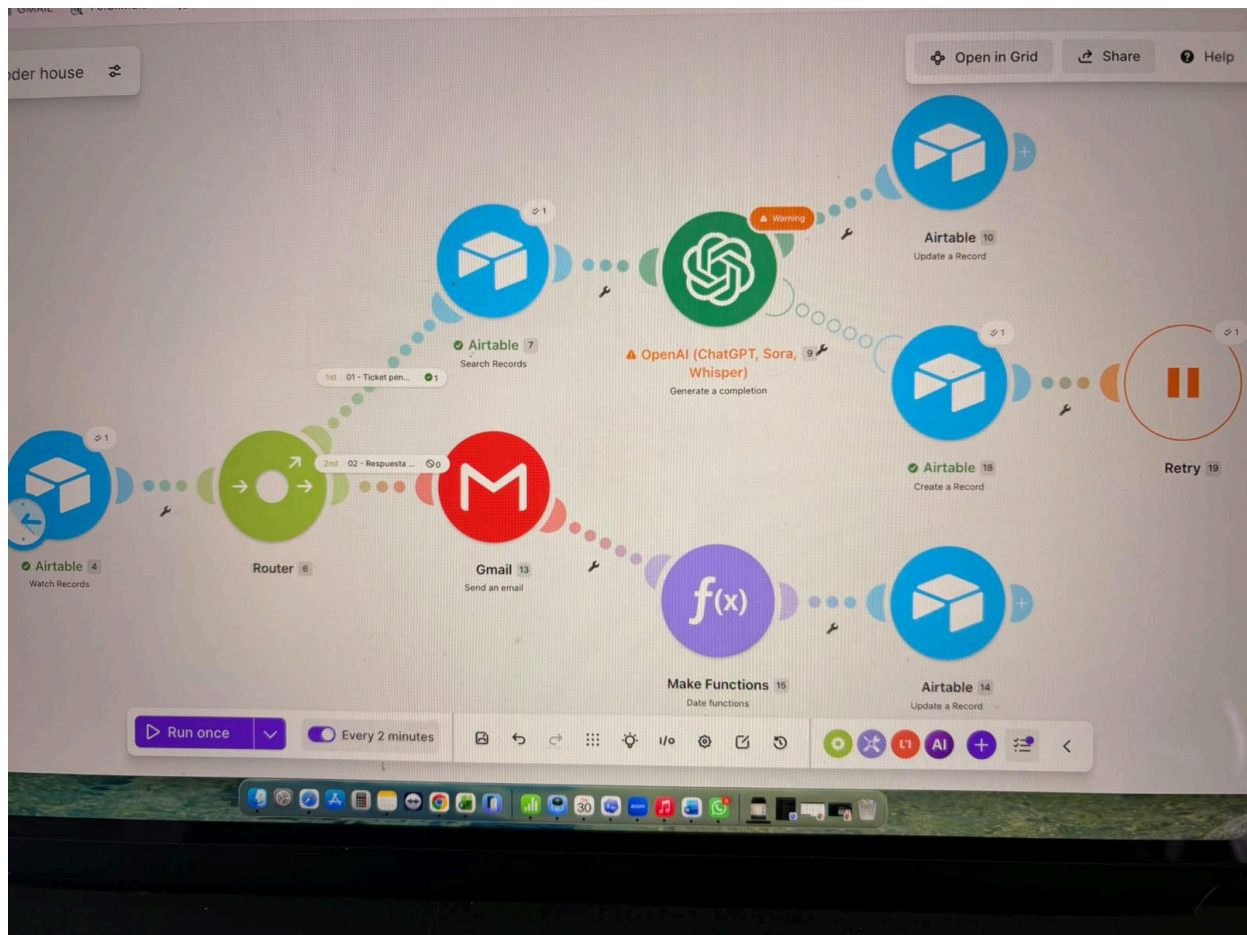
Esta tabla conserva la información técnica de las ejecuciones fallidas.

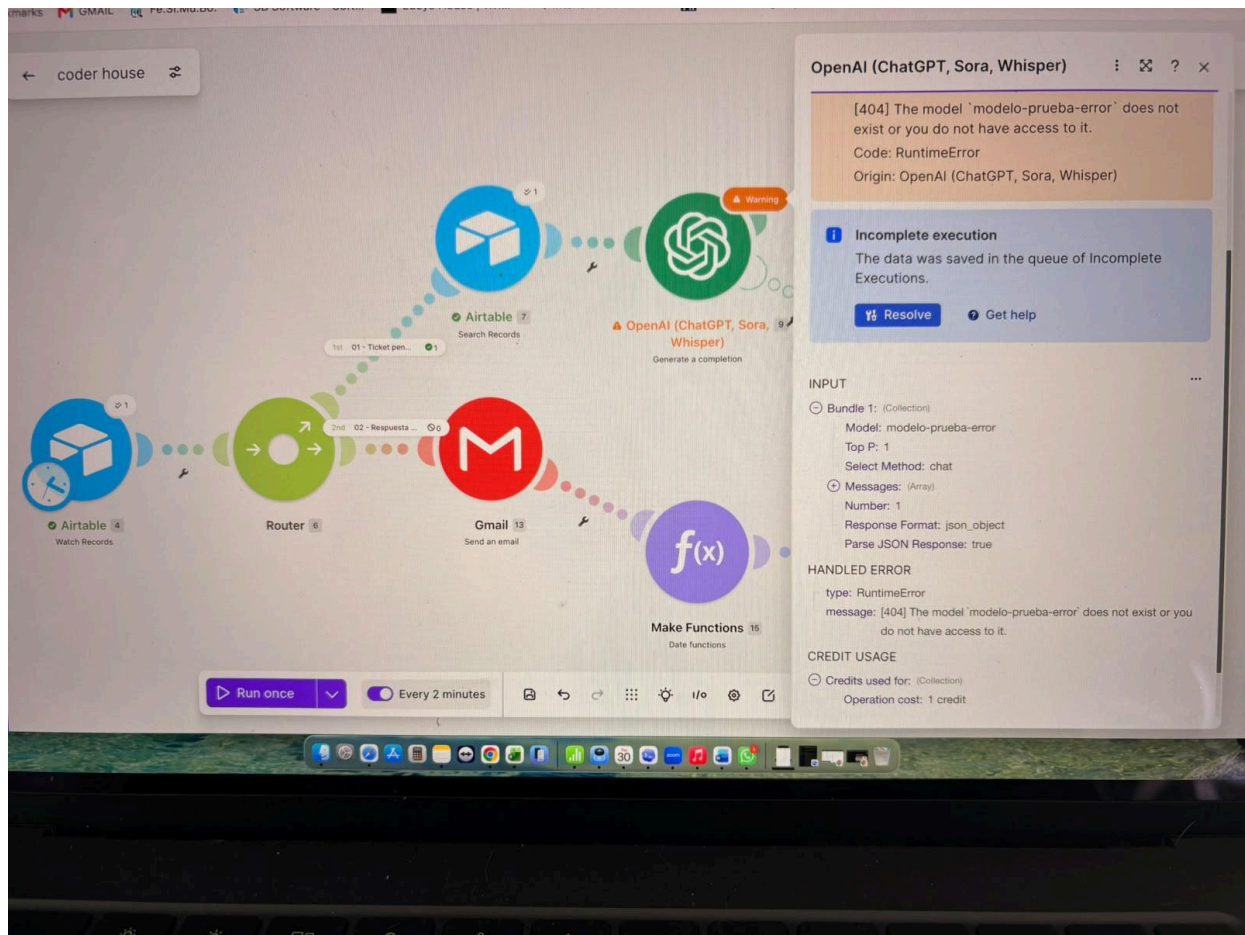
Campo	Función
ID del error	Identificador único, por ejemplo ERR-0001
Ticket relacionado	Vínculo con el ticket afectado

Escenario	Flujo donde ocurrió la falla
Módulo	Operación que produjo el error
Tipo de error	Clasificación técnica
Mensaje de error	Detalle devuelto por el servicio
Fecha y hora	Momento de la falla
Número de intentos	Control de reintentos
Estado del error	Pendiente, en revisión o resuelto
Diagnóstico	Explicación del problema detectado
Acción recomendada	Medida sugerida para resolverlo

Al vincular el registro técnico con el ticket, es posible conocer qué consulta fue afectada y evitar errores sin contexto.

Ejecución de la ruta de manejo de errores.





Registro técnico vinculado al ticket afectado.

RidMI									
Datos Automatizaciones Interfaces Formularios									
recursos de estudio prompt IA Estudiantes coder house proyecto Tickets coder house Registro de errores Configuración IA 1 tabla oculta Herramientas									
Grid view Ocultar campos Filtro Grupo Clasificar Configurar color Compartir y sincronizar									
ID error	Módulo	tipo de error	Código del error	Mensaje del error	Estado del error	Intentos	Fecha de resolución	Observaciones	
2	OpenAI	API		[404] The model 'modelo-prueba-error' does not exist or you do not have access to it.	Pendiente	1			

Ventajas del modelo relacional

La estructura implementada ofrece las siguientes ventajas:

- Evita repetir los datos de cada estudiante.
- Conserva el historial de tickets por usuario.
- Relaciona cada error con su ticket de origen.
- Centraliza la configuración del modelo.
- Facilita la actualización de parámetros sin modificar Make.

- Permite construir indicadores y gráficos.
- Mejora la trazabilidad de las decisiones de la IA y de la validación humana.
- Prepara la solución para incorporar nuevas categorías, canales o modelos en el futuro.

Esquemas JSON de transferencia de datos

Make funciona como intermediario entre Airtable, OpenAI y Gmail. Durante la ejecución, los datos son transformados y transferidos entre los distintos servicios mediante estructuras organizadas.

Los siguientes esquemas representan de forma simplificada la información intercambiada. Los valores son ilustrativos y se omiten identificadores sensibles, credenciales y claves de API.

Datos recuperados desde Airtable

Cuando el escenario detecta un ticket pendiente, recupera un conjunto de datos equivalente al siguiente:

```
{
  "ticket_id": "TKT-0014",
  "estado": "Pendiente",
  "asunto": "No puedo acceder a mi cuenta",
  "mensaje_original": "Intenté ingresar varias veces y no puedo acceder.",
  "estudiante": {
    "nombre": "Estudiante de prueba",
    "correo": "estudiante@ejemplo.com"
  }
}
```

Esta estructura contiene únicamente la información necesaria para clasificar la consulta, generar una respuesta y vincular el resultado con el estudiante correspondiente.

Configuración dinámica de inteligencia artificial

Antes de llamar a OpenAI, Make consulta la configuración activa almacenada en Airtable:

```
{
  "configuracion": "Soporte RiDMi",
  "activa": true,
  "modelo": "gpt-4o-mini",
  "temperatura": 0.2,
  "max_tokens": 500,
  "categorias_permitidas": [
    "Técnica",
```

```

    "Académica",
    "Administrativa",
    "Otra"
  ],
  "prioridades_permitidas": [
    "Alta",
    "Media",
    "Baja"
  ],
  "sentimientos_permitidos": [
    "Positivo",
    "Neutral",
    "Negativo"
  ]
}

```

El almacenamiento de estos parámetros en Airtable evita que el modelo, las categorías y los límites queden fijados permanentemente dentro del escenario.

Solicitud enviada a OpenAI

Make combina la configuración activa con los datos del ticket y construye una solicitud dinámica:

```

{
  "model": "gpt-4o-mini",
  "temperature": 0.2,
  "max_tokens": 500,
  "messages": [
    {
      "role": "system",
      "content": "Sos un asistente de soporte estudiantil de RiDMi. Analizá la consulta y devolvé únicamente un objeto JSON válido."
    },
    {
      "role": "user",
      "content": "Ticket: TKT-0014\nAsunto: No puedo acceder a mi cuenta\nMensaje: Intenté ingresar varias veces y no puedo acceder."
    }
  ]
}

```

Los campos `model`, `temperature`, `max_tokens`, `asunto` y `mensaje` se completan con datos obtenidos durante la ejecución. Por lo tanto, el prompt no depende de información fija asociada a un ticket específico.

Respuesta estructurada esperada de OpenAI

El modelo devuelve una respuesta con una estructura predefinida:

```
{
  "categoria": "Técnica",
  "prioridad": "Alta",
  "sentimiento": "Negativo",
  "respuesta_propuesta": "Hola. Lamentamos el inconveniente con el acceso. Te recomendamos verificar tu correo y solicitar el restablecimiento de contraseña. Si el problema continúa, respondé este mensaje para que podamos ayudarte."
}
```

El uso de una salida estructurada permite mapear cada propiedad hacia un campo diferente de Airtable.

Actualización del ticket después del procesamiento

Después de interpretar la respuesta, Make actualiza el registro original:

```
{
  "categoria_ia": "Técnica",
  "prioridad_ia": "Alta",
  "sentimiento_ia": "Negativo",
  "respuesta_propuesta": "Hola. Lamentamos el inconveniente con el acceso...",
  "estado": "Esperando aprobación"
}
```

El cambio de estado impide el envío inmediato y activa la etapa de revisión humana.

Datos enviados a Gmail

Cuando el ticket fue aprobado, Make construye el mensaje destinado al estudiante:

```
{
  "to": "estudiante@ejemplo.com",
  "subject": "Respuesta a tu consulta TKT-0014",
  "content": "Hola. Lamentamos el inconveniente con el acceso...",
  "ticket_id": "TKT-0014"
}
```

El correo del destinatario proviene de la relación entre las tablas **Tickets coder house** y **Estudiantes**. La respuesta utilizada debe ser la versión validada para el envío.

Resultado registrado después del envío

Gmail devuelve información de la operación. Make conserva los datos necesarios para la trazabilidad:

```
{
  "estado": "Enviado",
  "gmail_thread_id": "identificador_devuelto_por_gmail",
  "fecha_envio": "2026-07-31T15:30:00-03:00",
  "aprobado_por": "Valentina Morey",
  "fecha_aprobacion": "2026-07-31"
}
```

El **Gmail Thread ID** permite identificar la conversación y mantener organizados los mensajes relacionados.

Registro generado ante una falla

Si la solicitud a OpenAI falla, la ruta de error crea una estructura equivalente a la siguiente:

```
{
  "error_id": "ERR-0001",
  "ticket_relacionado": "TKT-0011",
  "escenario": "RiDMi - Soporte IA",
  "modulo": "OpenAI",
  "tipo_error": "API_ERROR",
  "codigo": 404,
  "mensaje": "The requested resource was not found",
  "numero_intentos": 1,
  "estado_error": "Pendiente",
  "fecha_hora": "2026-07-31T14:20:00-03:00",
  "diagnostico": "La solicitud utilizó una configuración inválida o un recurso no disponible.",
  "accion_recomendada": "Revisar el modelo, la conexión y la configuración del módulo."
}
```

Simultáneamente, el ticket afectado cambia al estado **Error**. De este modo, una falla de la API no produce un envío incorrecto ni queda oculta.

Mapeo entre servicios

Origen	Dato	Destino	Utilización
Airtable – Tickets	Asunto y mensaje	OpenAI	Interpretación de la consulta
Airtable – Configuración IA	Modelo y parámetros	OpenAI	Control del procesamiento

OpenAI	Categoría	Airtable – Tickets	Clasificación temática
OpenAI	Prioridad	Airtable – Tickets	Orden de atención
OpenAI	Sentimiento	Airtable – Tickets	Contexto emocional
OpenAI	Respuesta propuesta	Airtable – Tickets	Revisión humana
Airtable – Estudiantes	Correo	Gmail	Destinatario
Airtable – Tickets	Respuesta aprobada	Gmail	Contenido final
Gmail	Thread ID	Airtable – Tickets	Trazabilidad
Make – Error Handler	Información del error	Registro de errores	Diagnóstico y seguimiento

Los ejemplos anteriores no contienen credenciales, tokens de acceso ni claves privadas. Estos elementos permanecen protegidos dentro de las conexiones configuradas en Make.

Procesamiento mediante inteligencia artificial

OpenAI constituye el motor de interpretación del sistema. Su función no se limita a generar texto: analiza el lenguaje natural del estudiante y transforma una consulta libre en información estructurada que puede ser utilizada por la automatización.

Tareas realizadas por el modelo

En una única operación, el modelo debe:

1. Identificar la temática principal del ticket.
2. Asignar una categoría permitida.
3. Determinar el nivel de prioridad.
4. Analizar el sentimiento expresado.
5. Redactar una respuesta clara y contextualizada.
6. Devolver los resultados en un formato estructurado.
7. Evitar afirmar que realizó acciones que el sistema todavía no ejecutó.
8. Mantener un tono respetuoso y adecuado para estudiantes.

Prompt dinámico

El prompt utiliza variables recuperadas desde Airtable. Su estructura conceptual es la siguiente:

Actúa como asistente de soporte estudiantil de RiDMi.

Analizá el siguiente ticket:

ID: {{ID del ticket}}

Asunto: {{Asunto}}

Mensaje: {{Mensaje original}}

Clasificá la consulta utilizando únicamente estas opciones:

Categorías: {{Categorías permitidas}}

Prioridades: {{Prioridades permitidas}}

Sentimientos: {{Sentimientos permitidos}}

Generá una respuesta breve, clara, empática y orientada a resolver la consulta.

Devolvé únicamente un JSON válido con esta estructura:

```
{
  "categoria": "",
  "prioridad": "",
  "sentimiento": "",
  "respuesta_propuesta": ""
}
```

Los elementos encerrados entre llaves representan valores mapeados por Make. Cada ejecución utiliza el contenido real del ticket y la configuración activa del sistema.

Ventajas del prompt estructurado

El diseño utilizado permite:

- Mantener clasificaciones uniformes.
- Reducir respuestas difíciles de procesar.
- Separar los resultados en campos individuales.
- Modificar las categorías desde Airtable.
- Limitar la extensión de la respuesta.
- Reducir el consumo de tokens.
- Facilitar la revisión humana.
- Evitar instrucciones específicas para un único ticket.
- Mejorar la trazabilidad de los resultados.

Parámetros de control

La temperatura se mantiene en un valor bajo para priorizar consistencia frente a creatividad. El máximo de tokens limita la extensión de la respuesta y evita consumos innecesarios.

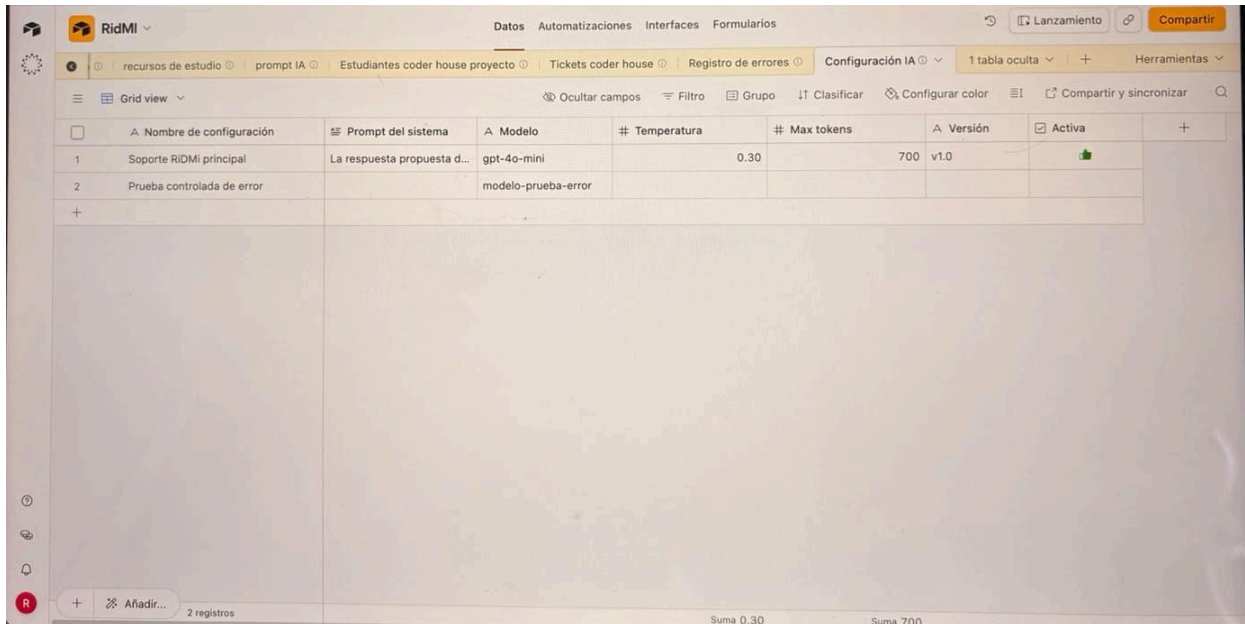
Estos parámetros se almacenan en la tabla **Configuración IA**, lo que permite ajustarlos sin modificar directamente la arquitectura de Make.

Limitaciones del procesamiento

Aunque el modelo permite agilizar la atención, puede interpretar incorrectamente una consulta, asignar una prioridad inadecuada o generar información imprecisa. Por este motivo, la respuesta se considera una propuesta y no una decisión definitiva.

La acción crítica —contactar al estudiante— permanece condicionada a una validación humana.

Variables y parámetros dinámicos utilizados para configurar el procesamiento con OpenAI.



The screenshot shows the RidMI application interface. At the top, there's a navigation bar with tabs: Datos, Automatizaciones, Interfaces, and Formularios. Below this is a breadcrumb trail: recursos de estudio > prompt IA > Estudiantes coder house proyecto > Tickets coder house > Registro de errores > Configuración IA. The main area displays a table with the following columns: Nombre de configuración, Prompt del sistema, Modelo, Temperatura, Max tokens, Versión, and Activa. There are two rows of data. The first row is for 'Soporte RIDMI principal' with a temperature of 0.30 and 700 max tokens. The second row is for 'Prueba controlada de error' with a model of 'modelo-prueba-error'. The table is currently in 'Grid view' and has 2 records. At the bottom, there are summary statistics: Suma 0.30 and Suma 700.

	Nombre de configuración	Prompt del sistema	Modelo	Temperatura	Max tokens	Versión	Activa
1	Soporte RIDMI principal	La respuesta propuesta d...	gpt-4o-mini	0.30	700	v1.0	☒
2	Prueba controlada de error		modelo-prueba-error				☐

Human-in-the-loop: validación humana

El sistema incorpora un punto obligatorio de control humano antes del envío del correo. Este mecanismo evita el denominado “efecto metralleta”, es decir, el envío automático de respuestas generadas sin supervisión.

Momento de la intervención

Después de que OpenAI clasifica el ticket y redacta la propuesta, Make guarda los resultados y cambia el estado a:

Esperando aprobación

En este punto, el procesamiento automático se detiene. Gmail todavía no recibe ninguna instrucción de envío.

Proceso de revisión

La persona responsable revisa:

- El mensaje original del estudiante.
- La categoría asignada.
- La prioridad detectada.
- El sentimiento identificado.
- La respuesta propuesta por la IA.
- La exactitud y pertinencia de la respuesta.
- La presencia de información sensible o inadecuada.

Si es necesario, puede corregir el contenido antes de autorizarlo.

Registro de la aprobación

La validación queda documentada mediante:

Estado: Aprobado por humano

Aprobado por: Valentina Morey

Fecha de aprobación: 29/07/2026 - 30/07/2026 - 31/07/2026

Estos campos permiten identificar quién autorizó la acción y cuándo se realizó la revisión.

Reanudación del flujo

En una ejecución posterior, Make detecta que el ticket tiene el estado **Aprobado por humano**. La ruta de envío recupera la respuesta autorizada, obtiene el correo relacionado y envía el mensaje mediante Gmail.

Finalmente, registra:

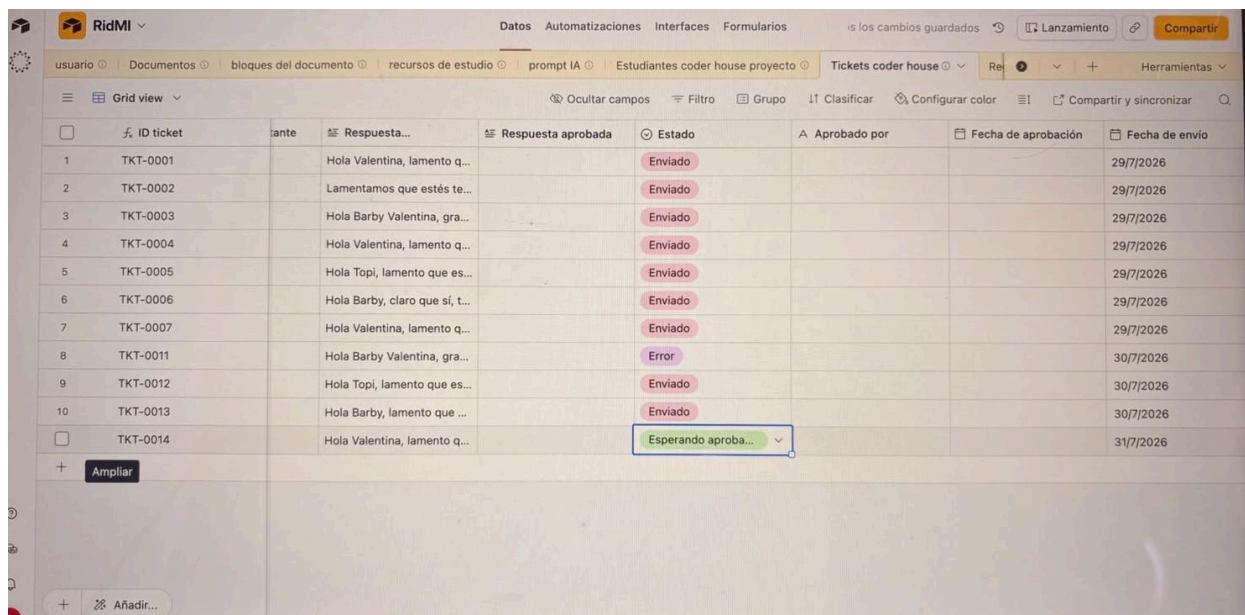
- Estado **Enviado**.
- Fecha de envío.
- **Gmail Thread ID**.
- Responsable de la aprobación.
- Fecha de aprobación.

Importancia del control humano

Este mecanismo:

- Reduce el riesgo de enviar información incorrecta.
- Evita respuestas ofensivas o fuera de contexto.
- Permite corregir interpretaciones de la IA.
- Conserva la responsabilidad humana sobre la comunicación.
- Mejora la confianza en el sistema.
- Aporta evidencia de auditoría.
- Protege la imagen institucional de RiDMi.

Ticket detenido en espera de revisión.



ID ticket	Respuesta...	Respuesta aprobada	Estado	Aprobado por	Fecha de aprobación	Fecha de envío
1 TKT-0001	Hola Valentina, lamento q...		Enviado			29/7/2026
2 TKT-0002	Lamentamos que estés te...		Enviado			29/7/2026
3 TKT-0003	Hola Barby Valentina, gra...		Enviado			29/7/2026
4 TKT-0004	Hola Valentina, lamento q...		Enviado			29/7/2026
5 TKT-0005	Hola Topi, lamento que es...		Enviado			29/7/2026
6 TKT-0006	Hola Barby, claro que sí, t...		Enviado			29/7/2026
7 TKT-0007	Hola Valentina, lamento q...		Enviado			29/7/2026
8 TKT-0011	Hola Barby Valentina, gra...		Error			30/7/2026
9 TKT-0012	Hola Topi, lamento que es...		Enviado			30/7/2026
10 TKT-0013	Hola Barby, lamento que ...		Enviado			30/7/2026
TKT-0014	Hola Valentina, lamento q...		Esperando aproba...			31/7/2026

Aprobación identificada y fechada antes del envío mediante Gmail.

ID ticket	Respuesta	Respuesta aprobada	Estado	Aprobado por	Fecha de aprobación	Fecha de envío
1 TKT-0001	Hola Valentina, lamento q...		Enviado			29/7/2026
2 TKT-0002	Lamentamos que estés te...		Enviado			29/7/2026
3 TKT-0003	Hola Barby Valentina, gra...		Enviado			29/7/2026
4 TKT-0004	Hola Valentina, lamento q...		Enviado			29/7/2026
5 TKT-0005	Hola Topi, lamento que es...		Enviado			29/7/2026
6 TKT-0006	Hola Barby, claro que sí, t...		Enviado			29/7/2026
7 TKT-0007	Hola Valentina, lamento q...		Enviado			29/7/2026
8 TKT-0011	Hola Barby Valentina, gra...		Error			30/7/2026
9 TKT-0012	Hola Topi, lamento que es...		Enviado			30/7/2026
10 TKT-0013	Hola Barby, lamento que ...		Enviado			30/7/2026
TKT-0014	Hola Valentina, lamento q...		Aprobado por hum...			31/7/2026

Datos utilizados por componente

Componente	Datos utilizados	Justificación
Airtable	Datos operativos del estudiante, tickets y resultados	Memoria y trazabilidad
Make	Datos necesarios para coordinar cada operación	Orquestación
OpenAI	Asunto y mensaje del ticket	Clasificación y generación de respuesta
Gmail	Correo, asunto y respuesta aprobada	Comunicación final
Dashboard	Estados, categorías, prioridades y errores	Monitoreo operativo

Protección de credenciales

Las claves y autorizaciones se administran mediante las conexiones internas de Make. No deben escribirse dentro de:

- Prompts.
- Campos de Airtable.
- Blueprint publicado.
- Capturas de pantalla.
- Video demostrativo.
- Repositorio de GitHub.
- Ejemplos JSON del informe.

Antes de publicar los archivos, debe comprobarse que no exista ninguna API Key, token, contraseña o encabezado de autorización visible.

Protección de datos personales

El sistema utiliza el nombre y correo electrónico únicamente cuando son necesarios para identificar al estudiante y enviar la respuesta.

Para la documentación pública se aplican las siguientes medidas:

- Evitar mostrar documentos de identidad.
- Utilizar datos ficticios o autorizados.
- Ocultar parcialmente correos reales cuando sea necesario.
- No publicar información académica sensible.
- Limitar los permisos de edición en Airtable.
- Compartir la base y el dashboard en modo lectura.
- Revisar las capturas antes de subirlas a GitHub.

Control de acceso

El acceso de edición debe mantenerse restringido a las personas responsables de operar el sistema. Los enlaces entregados para evaluación deben permitir la visualización, pero no la modificación de registros ni configuraciones.

Prevención de acciones incorrectas

La seguridad operativa también se sostiene mediante:

- Filtros por estado.
- Aprobación humana.

- Respuestas estructuradas.
- Categorías controladas.
- Límites de tokens.
- Registro de errores.
- Detención ante fallas.
- Trazabilidad del envío.
- Separación entre respuesta propuesta y respuesta autorizada.

Manejo de errores y resiliencia

La resiliencia consiste en mantener el sistema bajo control cuando una integración falla o recibe datos inválidos. El escenario incorpora una ruta específica asociada al módulo de OpenAI.

Falla probada

Para verificar el comportamiento del camino infeliz, se provocó de manera controlada un error **404** en la operación de OpenAI.

La prueba permitió comprobar que el escenario:

1. Detectó la falla de la API.
2. Evitó continuar hacia el envío.
3. Actualizó el ticket afectado al estado **Error**.
4. Creó un registro técnico en Airtable.
5. Relacionó el error con el ticket original.
6. Conservó el mensaje y el diagnóstico.
7. Permitió identificar una acción recomendada.

Información almacenada

El registro técnico conserva:

- Escenario afectado.
- Módulo que produjo la falla.
- Tipo y código de error.
- Mensaje devuelto por la API.
- Fecha y hora.
- Número de intentos.
- Estado de resolución.
- Diagnóstico.
- Acción recomendada.
- Ticket relacionado.

Esta información facilita el mantenimiento y evita depender exclusivamente del historial temporal de Make.

Directiva de control

La ruta utiliza una directiva de manejo de errores para reintentar o interrumpir la ejecución de manera controlada.

El reintento resulta adecuado para fallas temporales. La detención mediante **Break** se utiliza cuando continuar podría producir datos incompletos, duplicados o comunicaciones incorrectas.

Prevención de envíos posteriores al error

Cuando el ticket cambia a **Error**, deja de cumplir las condiciones de las rutas **Pendiente** y **Aprobado por humano**. Por lo tanto, no puede avanzar accidentalmente hacia Gmail.

Para volver a procesarlo se requiere revisar la causa, corregir la configuración y modificar conscientemente su estado.

Protección frente a datos faltantes

Los filtros y validaciones deben comprobar, como mínimo, la presencia de:

- ID del ticket.
- Asunto.
- Mensaje original.
- Estudiante relacionado.
- Correo electrónico.
- Configuración de IA activa.
- Respuesta aprobada antes del envío.

Si falta un dato indispensable, el registro no debe avanzar hacia una acción crítica.

Control de bucles

Los estados crean una secuencia unidireccional:

Pendiente

→ Esperando aprobación

→ Aprobado por humano

→ Enviado

La ruta alternativa es:

Pendiente

→ Falla de procesamiento

→ Error

→ Revisión técnica

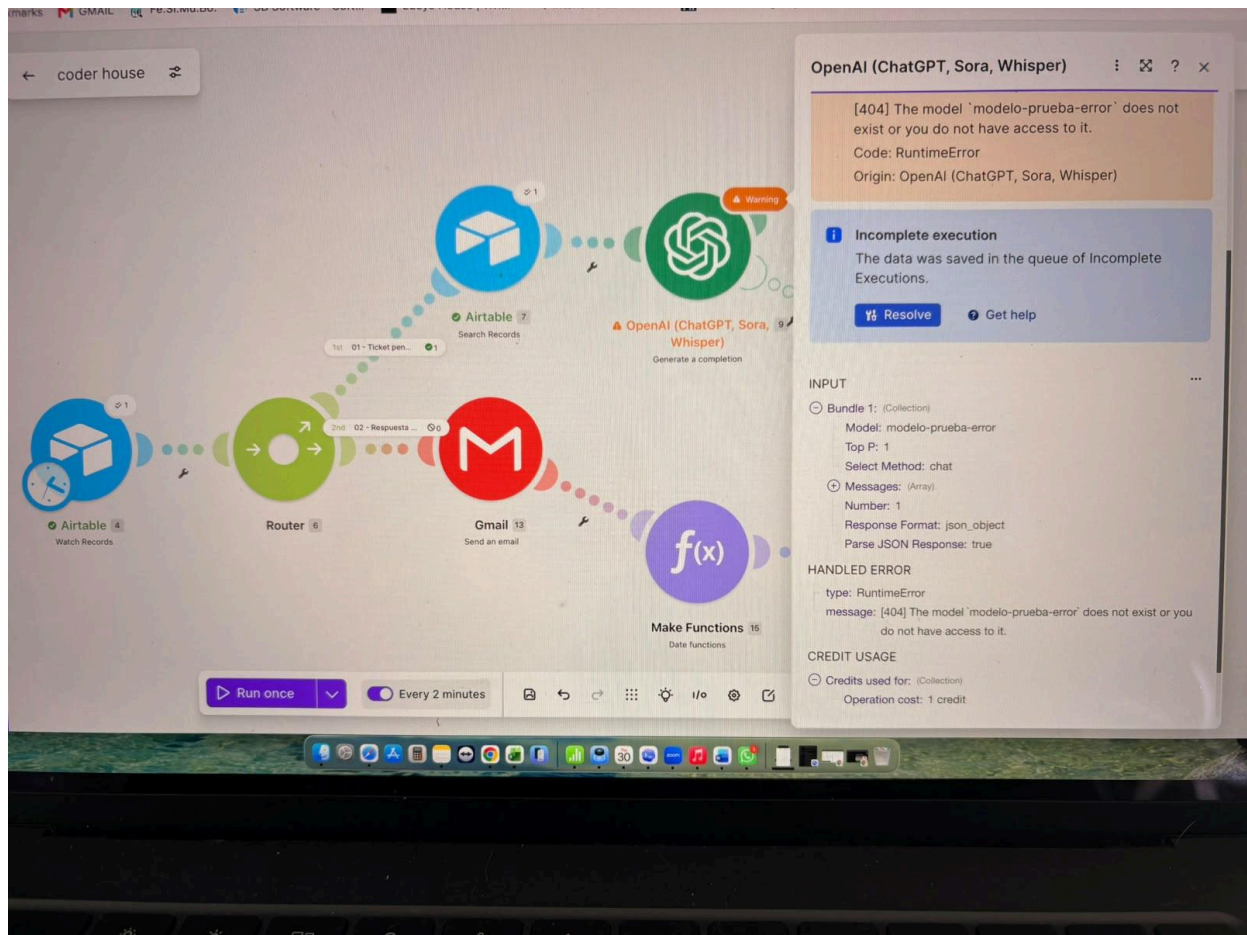
Cada operación modifica el estado del ticket. Así, el registro deja de cumplir la condición que inició la etapa anterior y se reduce el riesgo de ejecuciones repetidas.

Recuperación operativa

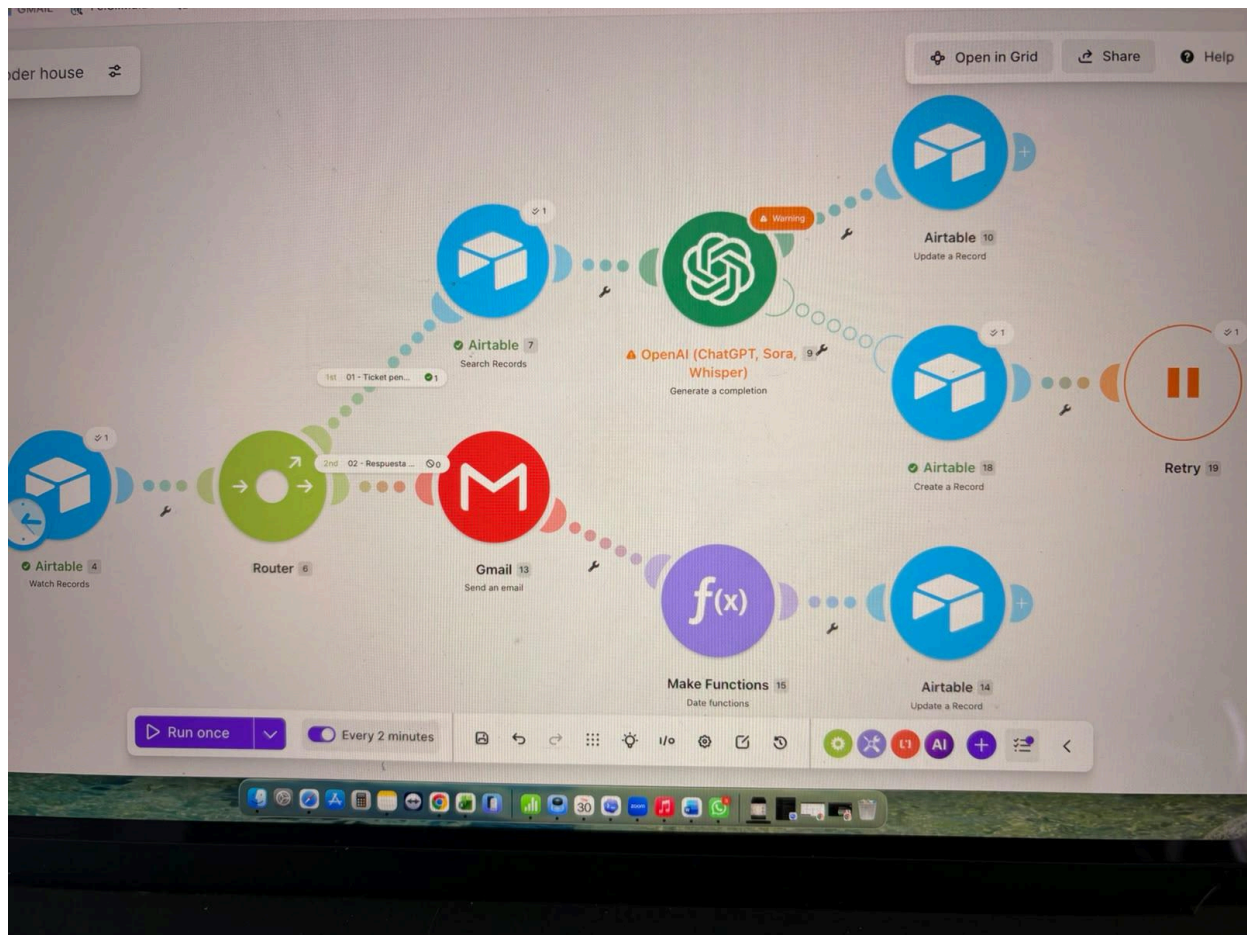
Ante una falla, la persona responsable debe:

1. Consultar el registro asociado.
2. Revisar el código y el mensaje del error.
3. Verificar la configuración activa.
4. Corregir la conexión, el modelo o los datos.
5. Registrar la acción realizada.
6. Cambiar el estado del error cuando corresponda.
7. Reprocesar el ticket de manera controlada.
8. Confirmar que no exista un correo enviado previamente.

Error controlado producido durante la prueba de resiliencia.



Ruta de Error Handler ejecutada en Make.



Registro persistente del error y relación con el ticket afectado

RidMI									
Datos Automatizaciones Interfaces Formularios									
recursos de estudio prompt IA Estudiantes coder house proyecto Tickets coder house Registro de errores Configuración IA 1 tabla oculta Herramientas									
Grid view Ocultar campos Filtro Grupo Clasificar Configurar color Compartir y sincronizar									
ID error	Módulo	tipo de error	Código del error	Mensaje del error	Estado del error	Intentos	Fecha de resolución	Observaciones	
2	OpenAI	API		[404] The model 'modelo-prueba-error' does not exist or you do not have access to it.	Pendiente	1			

Matriz comparativa y optimización de costos

La selección del modelo de inteligencia artificial influye directamente en el costo, la velocidad y la calidad de las respuestas. Para este proyecto se priorizó un modelo económico y suficientemente preciso para tareas breves de clasificación y redacción.

Criterios de selección

Los modelos se evaluaron considerando:

- Costo de procesamiento.
- Velocidad de respuesta.
- Capacidad para interpretar consultas en español.
- Seguimiento de instrucciones.
- Generación de JSON estructurado.
- Calidad de la redacción.
- Complejidad de la tarea.
- Escalabilidad para un volumen mayor de tickets.

Matriz comparativa

Alternativa	Costo relativo	Velocidad	Calidad esperada	Uso recomendado
Modelo pequeño o “mini”	Bajo	Alta	Adecuada para clasificación y respuestas breves	Operación habitual de tickets
Modelo general avanzado	Medio	Media/alta	Mayor comprensión de casos ambiguos	Consultas complejas o excepcionales
Modelo de razonamiento	Alto	Menor	Mayor profundidad analítica	Casos que requieran análisis especializado
Revisión completamente humana	Costo operativo alto	Variable	Depende de la persona responsable	Validación final y situaciones sensibles

Utilizar GPT-4o mini para el procesamiento habitual permite reducir significativamente el costo frente a un modelo general avanzado. Para 1.000 tickets, equivalentes aproximadamente a 500.000 tokens de entrada y 250.000 tokens de salida, el ahorro estimado supera el 90 %, según las tarifas comparadas al momento de elaborar el proyecto. Los valores deben considerarse ilustrativos, ya que los precios de la API pueden actualizarse.

Los precios de las APIs pueden cambiar con el tiempo. Por ese motivo, la comparación se plantea principalmente en términos relativos y la configuración del modelo se conserva en Airtable para permitir su sustitución sin reconstruir el escenario.

Como referencia de la etapa de implementación, se seleccionó un modelo de categoría “mini” porque la tarea no requiere razonamientos extensos: debe interpretar una consulta corta, asignar tres clasificaciones y redactar una respuesta breve.

Modelo seleccionado

El modelo configurado para el proyecto es:

gpt-4o-mini

La elección se justifica porque ofrece una relación favorable entre costo, velocidad y calidad para:

- Clasificar tickets.
- Analizar el sentimiento.
- Estimar la prioridad.
- Seguir un esquema JSON.
- Generar respuestas breves en español.
- Procesar un volumen creciente de consultas.

No sería eficiente utilizar un modelo de razonamiento avanzado para todos los tickets, debido a que la mayoría corresponde a consultas técnicas, administrativas o académicas de complejidad baja o media.

Estimación ilustrativa del consumo

Para analizar el costo se puede considerar un ticket promedio compuesto por:

Entrada aproximada: 500 tokens

Salida aproximada: 250 tokens

Total aproximado: 750 tokens por ticket

La estimación incluye el prompt del sistema, la configuración, el asunto, el mensaje original, la clasificación y la respuesta propuesta.

Volumen	Entrada estimada	Salida estimada	Total aproximado
100 tickets	50.000 tokens	25.000 tokens	75.000 tokens
1.000 tickets	500.000 tokens	250.000 tokens	750.000 tokens
10.000 tickets	5.000.000 tokens	2.500.000 tokens	7.500.000 tokens

El costo monetario exacto debe calcularse con la tarifa vigente del modelo al momento de operar el sistema. OpenAI publica sus precios actualizados en su [página oficial de precios](#).

Estrategias de optimización aplicadas

La arquitectura incorpora distintas medidas para controlar el consumo:

- Uso de un modelo pequeño para la operación habitual.
- Procesamiento exclusivo de tickets con estados válidos.
- Límite de tokens de salida.
- Temperatura baja.
- Prompt breve y específico.
- Una sola llamada de IA para clasificar y redactar.
- Salida JSON con propiedades determinadas.
- Exclusión de información innecesaria.
- Configuración dinámica desde Airtable.
- Separación entre procesamiento y envío.
- Prevención de reprocesamientos.
- Registro de errores para evitar reintentos indefinidos.

Estrategia de escalamiento

Si RiDMi aumenta su volumen de tickets, puede mantener el modelo económico para la mayoría de los casos y derivar solamente los tickets complejos hacia un modelo más avanzado.

La estrategia propuesta es:

Consultas frecuentes o simples

→ Modelo mini

Consultas ambiguas o sensibles

→ Revisión humana o modelo avanzado

Casos críticos

→ Intervención humana obligatoria

Este enfoque permite equilibrar costo y calidad sin disminuir la seguridad operativa.

Dashboard de control e indicadores

Se creó un dashboard en Airtable Interfaces para centralizar la supervisión del sistema. Su objetivo es transformar los registros operativos en información visual que facilite la toma de decisiones.

17.1. Indicadores principales

El dashboard incluye los siguientes KPIs:

- Cantidad total de tickets.
- Tickets pendientes.
- Tickets esperando aprobación.
- Tickets aprobados.
- Tickets enviados.
- Tickets con error.
- Tickets registrados recientemente.
- Distribución por categoría.
- Distribución por prioridad.
- Distribución por sentimiento.
- Registros técnicos pendientes de resolución.

17.2. Indicadores visuales

Indicador	Tipo de visualización	Finalidad
Total de tickets	Número o KPI	Medir el volumen general
Tickets enviados	Número o KPI	Conocer las respuestas completadas
Tickets recientes	Número o lista	Supervisar la actividad actual
Tickets con error	Número o KPI	Detectar problemas operativos
Tickets por categoría	Gráfico circular o de barras	Identificar los motivos más frecuentes
Tickets por prioridad	Gráfico de barras	Observar la urgencia de los casos
Tickets por sentimiento	Gráfico circular o de barras	Analizar el tono de las consultas
Errores por estado	Tabla o gráfico	Controlar fallas pendientes y resueltas

Tasa de errores

La tasa de errores permite conocer qué proporción de los tickets procesados presentó una falla técnica.

Se calcula mediante:

$$\text{Tasa de errores} = (\text{Cantidad de tickets con error} / \text{Total de tickets procesados}) \times 100$$

Por ejemplo, si se procesaron 20 tickets y uno terminó en estado **Error**:

$$(1 / 20) \times 100 = 5 \%$$

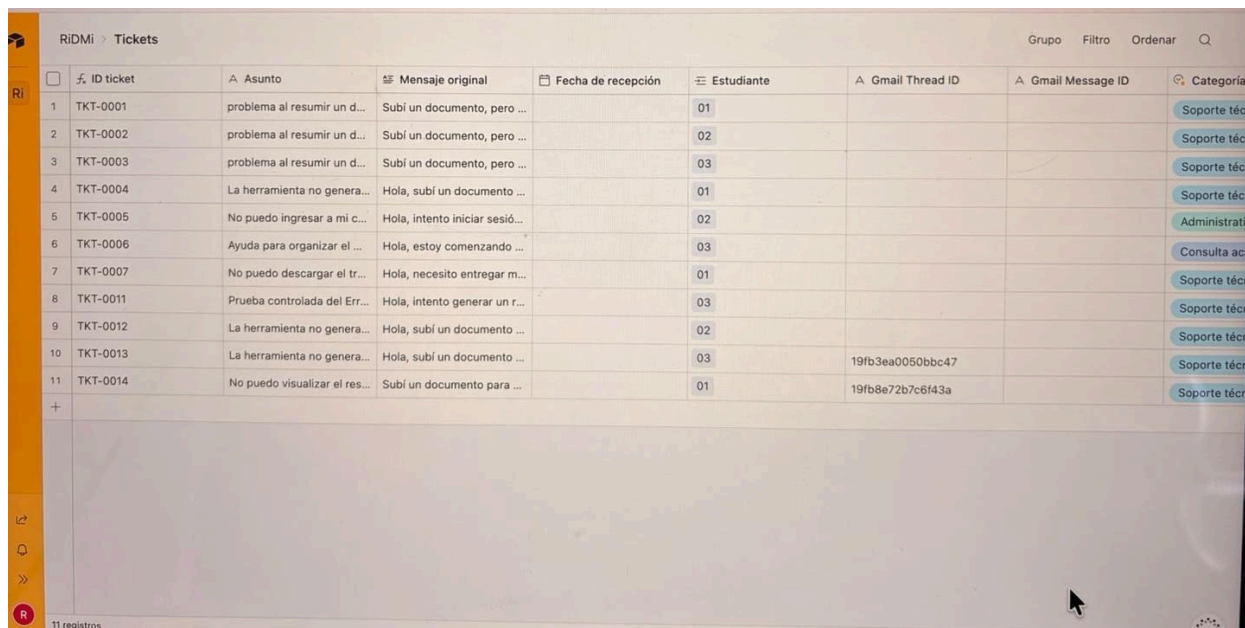
En este proyecto, el error observado durante las pruebas fue provocado deliberadamente para validar la ruta de resiliencia. Por lo tanto, debe diferenciarse un error controlado de prueba de una falla espontánea en producción.

Utilidad para la gestión

El dashboard permite:

- Detectar acumulaciones de tickets pendientes.
- Controlar qué respuestas esperan aprobación.
- Conocer las consultas más frecuentes.
- Identificar casos de prioridad alta.
- Supervisar fallas técnicas.
- Revisar la evolución del servicio.
- Evaluar la necesidad de ampliar categorías.
- Facilitar la toma de decisiones basada en datos.
- Presentar el funcionamiento del sistema sin ingresar al escenario de Make.

Vista general de indicadores del sistema.

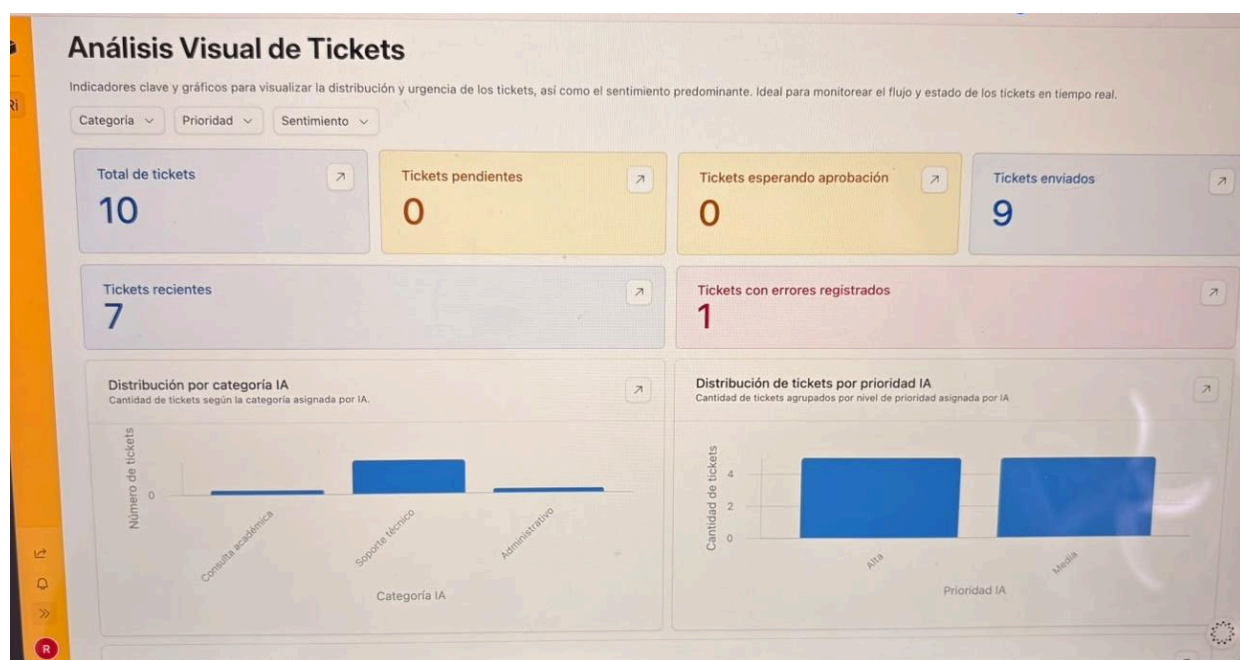


	ID ticket	Asunto	Mensaje original	Fecha de recepción	Estudiante	Gmail Thread ID	Gmail Message ID	Categoría
1	TKT-0001	problema al resumir un d...	Subí un documento, pero ...		01			Soporte técn
2	TKT-0002	problema al resumir un d...	Subí un documento, pero ...		02			Soporte técn
3	TKT-0003	problema al resumir un d...	Subí un documento, pero ...		03			Soporte técn
4	TKT-0004	La herramienta no genera...	Hola, subí un documento ...		01			Soporte técn
5	TKT-0005	No puedo ingresar a mi c...	Hola, intento iniciar sesió...		02			Administrati
6	TKT-0006	Ayuda para organizar el ...	Hola, estoy comenzando ...		03			Consulta ac
7	TKT-0007	No puedo descargar el tr...	Hola, necesito entregar m...		01			Soporte técn
8	TKT-0011	Prueba controlada del Err...	Hola, intento generar un r...		03			Soporte técn
9	TKT-0012	La herramienta no genera...	Hola, subí un documento ...		02			Soporte técn
10	TKT-0013	La herramienta no genera...	Hola, subí un documento ...		03	19fb3ea0050bbc47		Soporte técn
11	TKT-0014	No puedo visualizar el res...	Subí un documento para ...		01	19fb8e72b7c6f43a		Soporte técn

RIDMi > Tickets							Grupo	Filtro	Ordenar	Q
☐	de recepción	Estudiante	A Gmail Thread ID	A Gmail Message ID	Categoría IA	☒ ¿Reciente?	☒ ¿Error pendiente?			
1		01			Soporte técnico					
2		02			Soporte técnico					
3		03			Soporte técnico					
4		01			Soporte técnico					
5		02			Administrativo					
6		03			Consulta académica					
7		01			Soporte técnico					
8		03			Soporte técnico					
9		02			Soporte técnico					
10		03	19fb3ea0050bbc47		Soporte técnico					
11		01	19fb8e72b7c6f43a		Soporte técnico					
+										

11 registros

Dashboard Gráficos





Acceso al dashboard

Enlaces de acceso al sistema:

- Base o vista de datos en Airtable:

<https://airtable.com/appRTpcuXDpfXkOfd/shrnDzXuWCkt5vdyb>

- Guía visual / dashboard:

<https://airtable.com/appRTpcuXDpfXkOfd/shrLsjN2eNJ6SKEuj>

Pruebas funcionales y test de estrés

La solución fue evaluada mediante casos representativos del funcionamiento esperado y una prueba controlada de error. El objetivo fue verificar el recorrido completo desde el ingreso del ticket hasta su clasificación, aprobación, envío o detención.

Metodología

Cada prueba incluyó:

1. Creación o selección de un ticket.
2. Asignación del estado inicial correspondiente.
3. Ejecución del escenario.
4. Revisión de la clasificación generada.

5. Verificación del cambio de estado.
6. Control de la intervención humana cuando correspondía.
7. Comprobación del envío o del registro del error.
8. Revisión de los resultados en Airtable.

Matriz general de pruebas

Prueba	Entrada o condición	Resultado esperado	Resultado obtenido	Estado
1	Ticket técnico urgente	Clasificación técnica, prioridad alta y respuesta propuesta	El ticket fue clasificado y quedó esperando aprobación	Exitoso
2	Consulta académica	Categoría académica y respuesta contextualizada	La categoría y la respuesta se guardaron correctamente	Exitoso
3	Problema administrativo	Clasificación administrativa	El modelo identificó la categoría correspondiente	Exitoso
4	Ticket aprobado	Envío por Gmail y registro del Thread ID	El correo se envió y la trazabilidad quedó registrada	Exitoso
5	API configurada para producir un error 404	Detención y creación de registro técnico	El flujo no envió el correo y guardó el error relacionado	Exitoso

Prueba 1: ticket técnico urgente

Se ingresó un ticket relacionado con un problema de acceso a la plataforma. El contenido expresaba que el estudiante había intentado ingresar varias veces sin éxito.

El sistema debía:

- Reconocer una consulta técnica.
- Asignar prioridad alta.
- Detectar un sentimiento negativo.
- Generar una respuesta empática.
- Cambiar el estado a **Esperando aprobación**.

El resultado fue exitoso. Los valores generados se almacenaron en sus campos correspondientes y no se produjo ningún envío automático.

Prueba 2: consulta académica

Se utilizó una consulta relacionada con el uso o la comprensión de una herramienta académica de RiDMi.

El sistema debía:

- Clasificarla como académica.
- Asignar una prioridad coherente.
- Interpretar el tono del estudiante.
- Generar una orientación breve.
- Detenerse antes del envío.

La respuesta se generó correctamente y conservó un tono apropiado para el público estudiantil.

Prueba 3: problema administrativo

Se registró una consulta vinculada con una gestión administrativa.

El modelo debía diferenciarla de un inconveniente técnico y de una solicitud académica. La categoría administrativa fue asignada correctamente y la respuesta propuesta se almacenó para su revisión.

Esta prueba permitió comprobar que el sistema no utiliza una única respuesta genérica para todas las consultas.

Prueba 4: aprobación y envío por Gmail

Se seleccionó un ticket previamente procesado y se completaron los campos de aprobación humana.

El estado se modificó a:

Aprobado por humano

En la siguiente ejecución, Make:

1. Detectó el nuevo estado.
2. Recuperó la respuesta autorizada.
3. Obtuvo el correo del estudiante.
4. Envío el mensaje mediante Gmail.
5. Registró el identificador del hilo.
6. Guardó la fecha de envío.
7. Cambió el estado a **Enviado**.

La prueba confirmó que Gmail solo interviene después de la aprobación.

Prueba 5: camino infeliz y error controlado

Para comprobar la resiliencia se provocó intencionalmente un error 404 en la integración con OpenAI.

El resultado esperado era:

- Capturar la falla.
- Impedir que el escenario avanzara.
- Actualizar el ticket.
- Crear un registro técnico.
- Relacionar el error con el ticket.
- Conservar el diagnóstico.
- Evitar el envío por Gmail.

La ruta se ejecutó correctamente. El error quedó registrado en Airtable y el flujo se detuvo de manera controlada.

Resultado del test de estrés funcional

Las cinco pruebas cubrieron:

- Distintas categorías.
- Diferentes niveles de prioridad.
- Generación de respuestas.
- Persistencia de los datos.
- Aprobación humana.
- Envío por correo.
- Trazabilidad.
- Manejo de errores.
- Prevención de envíos incorrectos.

Todas las pruebas alcanzaron el resultado esperado dentro del alcance definido.

Resultados obtenidos

La implementación permitió construir un ecosistema de automatización funcional que integra una base de datos, un orquestador, inteligencia artificial, validación humana, correo electrónico y monitoreo visual.

Resultados funcionales

El sistema logró:

- Detectar tickets según su estado.

- Recuperar configuraciones dinámicas.
- Analizar mensajes escritos en lenguaje natural.
- Clasificar consultas por categoría.
- Asignar niveles de prioridad.
- Identificar el sentimiento.
- Generar respuestas propuestas.
- Guardar cada resultado en Airtable.
- Detener el flujo para solicitar aprobación.
- Permitir la edición humana de la respuesta.
- Enviar únicamente los contenidos autorizados.
- Registrar fecha, responsable y **Gmail Thread ID**.
- Capturar fallas de OpenAI.
- Crear registros técnicos vinculados.
- Evitar que tickets con error lleguen a Gmail.
- Presentar indicadores en un dashboard.

Impacto operativo

En comparación con un procedimiento completamente manual, la solución reduce el trabajo repetitivo asociado con:

- Lectura inicial de cada consulta.
- Clasificación temática.
- Identificación de urgencia.
- Redacción de un primer borrador.
- Actualización de estados.
- Registro del envío.
- Documentación de errores.
- Elaboración de indicadores básicos.

La revisión humana continúa siendo necesaria, pero se concentra en validar y corregir, en lugar de comenzar cada respuesta desde cero.

Calidad y control

El sistema combina automatización y supervisión. OpenAI aporta velocidad y capacidad de interpretación, mientras que Airtable y Make proporcionan control del proceso.

La separación entre **Respuesta propuesta** y **Respuesta aprobada** garantiza que el contenido generado no sea tratado automáticamente como definitivo.

La información registrada permite reconstruir:

- Qué ticket ingresó.
- Qué clasificación recibió.

- Qué respuesta se propuso.
- Quién la aprobó.
- Cuándo fue enviada.
- Qué hilo generó Gmail.
- Qué error ocurrió, si existió.

Cumplimiento de los objetivos

Objetivo	Resultado
Centralizar la información	Cumplido mediante Airtable
Detectar tickets automáticamente	Cumplido mediante Watch Records
Incorporar inteligencia artificial	Cumplido mediante OpenAI
Utilizar configuración dinámica	Cumplido mediante la tabla Configuración IA
Clasificar y redactar	Cumplido
Mantener control humano	Cumplido
Enviar respuestas aprobadas	Cumplido mediante Gmail
Registrar trazabilidad	Cumplido
Manejar errores	Cumplido mediante Error Handler
Evitar bucles	Cumplido mediante filtros y estados
Visualizar indicadores	Cumplido mediante Airtable Interfaces
Probar el camino feliz e infeliz	Cumplido

Valor para RiDMi

La solución constituye una base aplicable al futuro sistema de soporte de RiDMi. Su arquitectura modular permite comenzar con un volumen reducido y ampliar gradualmente la operación.

Además de mejorar la atención, el sistema genera información útil para identificar problemas recurrentes, comprender las necesidades de los estudiantes y priorizar mejoras en la plataforma.

Limitaciones y mejoras futuras

La versión desarrollada cumple los objetivos del proyecto, pero presenta limitaciones propias de un prototipo funcional.

Limitaciones actuales

- La aprobación humana se realiza mediante cambios de estado en Airtable.
- El sistema utiliza Gmail como único canal de salida.
- No procesa automáticamente las respuestas posteriores del estudiante.
- Las categorías dependen de la configuración definida.
- La calidad de la clasificación depende de la claridad del mensaje.
- No existe una base de conocimiento especializada conectada al modelo.
- El error probado se concentra en la integración con OpenAI.
- El dashboard refleja los datos existentes, pero no realiza predicciones.
- El reprocesamiento de errores requiere intervención manual.
- No se implementó una gestión avanzada de roles y permisos.
- No se midieron tiempos de respuesta con un volumen masivo de tickets simultáneos.
- El sistema todavía no está integrado directamente con la plataforma web de RiDMi.

Mejoras de corto plazo

En una siguiente versión se podría:

- Crear un formulario específico para el ingreso de tickets.
- Incorporar botones de aprobar, corregir o rechazar.
- Agregar una notificación cuando un ticket espera revisión.
- Alertar sobre tickets de prioridad alta.
- Validar automáticamente campos obligatorios.
- Añadir más rutas de manejo de errores.
- Registrar la duración y el costo estimado de cada ejecución.
- Crear vistas específicas para tickets pendientes y vencidos.
- Incorporar plantillas de respuesta por categoría.
- Mejorar la anonimización de datos utilizados en pruebas.

Mejoras de mediano plazo

También se propone:

- Integrar el sistema con Softr o con la interfaz definitiva de RiDMi.
- Permitir que los estudiantes creen tickets desde su cuenta.
- Recibir y registrar respuestas dentro del mismo hilo.
- Incorporar acuerdos de nivel de servicio o SLA.
- Escalar automáticamente los casos críticos.
- Derivar tickets complejos a una persona especializada.
- Añadir búsqueda en documentación interna.
- Utilizar una base de conocimiento para responder con fuentes verificadas.

- Crear informes mensuales de rendimiento.
- Analizar tendencias y problemas recurrentes.

Mejoras avanzadas

En una etapa posterior, el ecosistema podría evolucionar hacia:

- Soporte multicanal mediante correo, chat y mensajería.
- Clasificación automática con niveles de confianza.
- Derivación a modelos diferentes según la complejidad.
- Detección de tickets duplicados.
- Resumen automático de conversaciones extensas.
- Evaluación de calidad de las respuestas.
- Encuestas de satisfacción posteriores al cierre.
- Paneles diferenciados por rol.
- Resolución automática de casos frecuentes de bajo riesgo.
- Sistema de auditoría y control de versiones de prompts.

Condiciones para una implementación productiva

Antes de utilizar el sistema con estudiantes reales se recomienda:

1. Revisar la política de privacidad.
2. Definir responsables y permisos.
3. Utilizar datos mínimos.
4. Probar un volumen mayor de tickets.
5. Documentar los procedimientos de recuperación.
6. Establecer tiempos máximos de atención.
7. Ampliar las rutas de error.
8. Revisar periódicamente los prompts.
9. Medir precisión, costo y satisfacción.
10. Mantener la aprobación humana en situaciones sensibles.

Proyección

El prototipo demuestra que es posible automatizar una parte importante del soporte de RiDMi sin perder control sobre la comunicación.

La arquitectura permite avanzar progresivamente: primero como asistente de clasificación y redacción; posteriormente como sistema integrado de soporte; y, finalmente, como ecosistema multicanal con una base de conocimiento propia.

Esta evolución deberá mantener como principios centrales la trazabilidad, la privacidad, el control de costos y la supervisión humana.

Conclusión

El proyecto permitió diseñar e implementar un ecosistema de automatización para la gestión de consultas estudiantiles de RiDMi, integrando Airtable, Make, OpenAI y Gmail.

La solución desarrollada recibe los tickets, interpreta su contenido, los clasifica por categoría, prioridad y sentimiento, genera una propuesta de respuesta y almacena los resultados para su revisión. Antes de enviar cualquier comunicación, incorpora una instancia obligatoria de aprobación humana, garantizando que la decisión final permanezca bajo supervisión.

También se implementaron mecanismos de trazabilidad, control de estados, prevención de ejecuciones duplicadas y manejo de errores. La prueba controlada de una falla permitió demostrar que el sistema puede detener el procesamiento, evitar envíos incorrectos y conservar información útil para el diagnóstico.

El dashboard desarrollado en Airtable facilita la supervisión de tickets, estados, prioridades, categorías y errores. Esto convierte los datos operativos en información útil para la gestión y permite identificar oportunidades de mejora en la atención a estudiantes.

La arquitectura propuesta es modular y escalable. Puede comenzar como un sistema de asistencia para un volumen reducido de consultas y evolucionar hacia una solución integrada con la plataforma RiDMi, nuevos canales de comunicación y una base de conocimiento especializada.

En conclusión, el proyecto demuestra que la inteligencia artificial puede reducir tareas repetitivas y agilizar la atención sin eliminar el control humano. La combinación de automatización, supervisión, trazabilidad y protección de datos permite construir un sistema funcional, responsable y adaptable a las futuras necesidades de RiDMi.

Fuentes y documentación consultada

- Airtable. Documentación sobre bases, automatizaciones e interfaces:
<https://support.airtable.com/>
- Make. Documentación oficial de la plataforma de automatización:
<https://help.make.com/>
- OpenAI. Documentación oficial de la API:
<https://platform.openai.com/docs/>
- OpenAI. Precios de utilización de la API:
<https://openai.com/api/pricing/>
- Gmail para desarrolladores. Documentación oficial:
<https://developers.google.com/gmail/api>

Anexos y enlaces de acceso

Vista de datos

<https://airtable.com/appRTpcuXDpfXkOfd/shrnDzXuWCkt5vdyb>

Dashboard o guía visual

<https://airtable.com/appRTpcuXDpfXkOfd/shrLsjN2eNJ6SKEuj>

Video demostrativo

Ver Video Demo de RiDMi

<https://drive.google.com/file/d/1JryK7pMXJeh9euJNeDUWyf4Y5nKcbfKz/view?usp=sharing>

Repositorio público de GitHub

<https://github.com/ridmiapp-design/ecosistema-automatizacion-ia-ridmi>